

Requirement Understanding

VA Disability Claims Product — Requirements Doc

1. Problem Statement

The VA disability claims process is a ~80+ day, multi-actor pipeline that is confusing for veterans and inefficient for the people who help them file (VSOs, attorneys, claims agents). Two structural failures drive most of the pain:

- Late validation. The people who know what evidence a rateable claim needs (raters) are downstream of everyone who assembles the evidence (veterans, VSOs, VSRs). ~40% of claims that reach rating get deferred and bounced back for more evidence — the single dominant rework loop in the system.
- No shared visibility. Veterans, VSOs, and VA caseworkers are all working the same claim with no shared, structured view of its state. Development requests go to the veteran by letter only — never to the VSO, even when the VSO holds Power of Attorney and can see the request sitting in the eFolder. Everyone is triangulating from partial information.

Three companies (one direct competitor, two useful analogs) map the shape of the opportunity — confirmed by reviewing each site directly (8/28/26):

- *Assured (assured.com)* — insurance claims, different vertical, not a VA competitor. Positions itself as "frontier AI for claims," an agentic orchestration layer across the P&C claims lifecycle: FNOL (first notice of loss) intake, a voice-AI agent, an agent named "Emma" that gathers info and recommends next-best-actions to human adjusters, fraud detection, and omnichannel messaging. Explicitly model-agnostic and pitches itself as "agents where they accelerate the work and determinism where it protects the outcome" — i.e., not full automation, human-in-the-loop where it counts, with claims-tuned evals and monitoring. This is the structural pattern worth borrowing: guided intake → structured, AI-summarized case view for the human decision-maker → deterministic checks doing the boring/high-stakes parts, agentic AI doing the informational parts.
- *PageAI / "Page" (pageai.co)* — the closest direct comparable, and confirmed to be exactly what the chat described: a backend-only platform sold to VA disability law firms (CCK, Jan Dils, Bergmann & Moore, and ~a dozen+ other named firms are customers). No veteran-facing front end at all — a firm uploads a veteran's C-file, and Page OCRs/organizes it (including handwritten records), summarizes it by condition or chronologically, lets attorneys query it in plain language ("AskPage"), and helps draft appeal documents. Two stages are explicitly marked "coming soon": an intake stage at the front of the workflow, and an "Analysis" stage that surfaces potential appeal opportunities and a case strategy. This is a meaningful signal: Page's own roadmap is moving toward the two things — intake and evidence-gap/opportunity analysis — that are core to our proposed wedge, meaning we should expect this gap to close over time and treat our timeline with urgency. Page is SOC 2 Type II, HIPAA-compliant, and explicitly frames itself around ABA

Formal Opinion 512 (lawyers must be able to verify AI output) — worth matching if we ever sell to attorneys.

- *VeteranHQ* (*veteranhq.app*) — originally flagged as just a "veteran-perspective guide," but on review it is itself a live, direct-to-veteran competing product, not just content. It publishes SEO-driven educational guides (e.g., "VA Disability Compensation Explained") that funnel into a registration flow ("Discover Your Benefits" → sign up), plus free tools (combined-rating calculator, conditions library, glossary) and a paid tier (has a pricing page and a Chrome extension). It explicitly does not file or prepare claims ("We educate and help you organize your evidence. We never prepare or file your claim for you") — it positions itself as an education/benefits-discovery layer that hands off to free accredited reps. This means VeteranHQ owns today's veteran-facing top-of-funnel (SEO content + benefit-matching) but deliberately stops short of the actual filing/tracking/adjudication workflow.

Net competitive picture:

Company	Veteran-facing UX	Structured case view for adjudicator	AI in the middle	Files/tracks the actual claim
Assured (different vertical)	Yes (insurance)	Yes	Yes — agentic + deterministic	Yes (insurance)
PageAI	No	Yes (attorney-facing)	Yes (OCR/summarize/draft)	Partial — drafts, doesn't file; "intake" and "analysis" stages are roadmap, not shipped
VeteranHQ	Yes (education/discovery only)	No	Light (calculators, matching)	No — explicitly declines to file/prepare
Us (proposed)	Yes	Yes (VSO-first)	Yes	TBD — open question, see Section 5

Our opportunity: No current player owns the veteran-facing intake/tracking experience and the adjudicator-facing (VSO-first) structured case tool and an AI layer doing the deterministic evidence-matching work in the middle, for VA disability specifically. PageAI is the entity most likely to close this gap on its own timeline (its "coming soon" intake + analysis stages point directly at it) — that's a reason to move, not a reason to avoid the space.

1.1 Why now

A separate, more current briefing (compiled 28 Aug 2026, drawing on VA/VBA public data plus roughly 200 practitioner interviews with VSOs, VA staff, attorneys, and veterans) sharpens the timing case considerably:

- Demand exploded. The 2022 PACT Act expanded eligibility to millions of veterans; claim volume has set records two years running, and ~40% of recent claims cite a PACT Act condition.
- The paid predators just died. The three largest unaccredited "claim shark" companies were hit with bankruptcy and court injunctions in 2026 (a ~\$250M/yr market of veterans who wanted to pay for help now has nowhere to go except the already-overloaded free accredited channel).
- Congress is actively picking a winner. Competing bills would either legalize or criminalize paid claims consultants; another would embed accredited reps directly into every military transition classroom. Every version increases the importance of the free, accredited VSO channel — which is exactly the channel our GTM already targets (Section 3.1).
- The rails are being replaced this year. A new, shorter claim form becomes mandatory Dec 31, 2026; DoD medical records are moving to same-day transfer at intent-to-file; VA's new Accredited Representative Portal (ARP) is absorbing all rep traffic and phasing out the older vendor integration (D2D) within roughly a year. Whoever builds the best layer on the new rails, not the old ones, wins the rep channel.
- VA itself is short-staffed and automating. VBA has cut roughly 30,000 positions including ~2,700 claims processors, while deploying more AI at the evidence-review step. A claim that arrives "automation-ready" — complete, correctly classified, well-organized — is now valuable to VA's own systems, not just to the veteran.

The updated headline metrics (Aug 2026) also sharpen Section 2's numbers considerably — see Section 2.11.

2. Background: How the System Actually Works Today

This section is the shared mental model the team needs before scoping the product. Full detail lives in the source field guide; this is the compressed version.

2.1 The pipeline, in one paragraph

The VA disability claims system is a request-processing platform: a messy ingress layer (everything from clean API payloads to faxed paper) → a trust boundary ("the firewall") where nearly everything gets flattened into PDF images → an internal ETL-plus-workflow engine that turns those images back into data → a human-in-the-loop adjudication pipeline. The system of record is VBMS/eFolder. A claim is "established" once it exists there, and "finished" once it's rated and authorized.

2.2 Trust zones

Zone	Who / what lives here
------	-----------------------

External	Veterans, VSOs, accredited attorneys/claims agents, unaccredited "claim sharks," private medical providers, DoD data feeds
Boundary	VA.gov 526 experience, Lighthouse API platform, QuickSubmit, ARP (Accredited Representative Portal), vendor mail/conversion services, exam vendors
Internal	Central Mail Portal, VBMS (Core/Rating/Awards), eFolder, CorpDB, NWQ (National Work Queue), ADS automation platform

Two deliberate holes let data cross inward automatically: DoD feeds (DEERS/VADIR) and read-only VSO access to the eFolder. Everything else that crosses the boundary is converted to a PDF and OCR'd back into data — including submissions that started as structured JSON.

2.3 Intake paths (how a claim gets in)

Four client types hit the same backend: VA.gov self-service (526 wizard — the only path that writes directly into VBMS as data), VSO/attorney software integrating via Lighthouse (D2D, roughly 14 different commercial/bespoke packages, the two biggest being VetPro and VetraSpec), a call-center agent, or a VA field employee. 60–68% of receipts still arrive as mail/fax despite a digitally-native veteran population — paper has no guided interview, no validation, just a form.

RPA-driven Mail Automation establishes ~74% of mailed/faxed/uploaded items successfully; the rest fall to human Claims Assistants to key in manually.

2.4 Development ("iDev") — where most of the ~77–80 day cycle lives

The VSR runs a ~14-point identity/service/POA/contention verification, then sets tracked items (the claim's de facto state machine) — each an outstanding dependency (an exam, private records, a medical opinion) with a suspense date (typically 30 days). While anything is outstanding, the claim is off-station, sitting in the NWQ cloud, and often gets picked up by a different VSR when it comes back. Claims typically pass through 2–3 VSRs. VSRs work a quota of ~10–14 claims/day.

Critically: development letters go to the veteran only, never the VSO — even when the VSO holds POA and can already see the tracked item in the eFolder. There is no VA→representative callback channel.

2.5 Rating, authorization, and the deferral loop

When development completes, an RVSR (Rating VSR) applies the VA Schedule for Rating Disabilities (VASRD); every decision must cite specific supporting evidence from the eFolder. An authorizer finalizes payment math and effective dates (this is where an Intent-to-File / Form 0966 timestamp matters — it backdates the payment lock). ~40% of claims that reach rating get deferred back to development — because the wrong exam was ordered, a clinician left required DBQ fields blank, or the evidence is ambiguous enough to require a formal medical opinion (adds ~a month).

2.6 Claim-type matters — the evidence ladder

The same intake form (526, or a decision-review form) carries very different validation rules depending on claim type:

Claim type	What the claimant must supply	What VA does
Original claim	Identity + contentions; evidence optional	Full duty-to-assist development
Claim for increase	Name + contention	Auto-orders exam, or skips via ACE (Acceptable Clinical Evidence)
Presumptive (PACT Act, ~2 dozen conditions)	The contention only	Skips service-connection development entirely
Supplemental claim	New/relevant evidence — mandatory	Re-decides with new evidence
Higher-level review	Argument only, no new evidence	Senior reviewer re-reads existing record
Fully Developed Claim (FDC)	Everything, certified complete	Skips development entirely

This distinction is, per the field guide, "the single most misunderstood part of the domain — the source of most veteran confusion." A veteran or VSO not understanding which lane they're in is a major source of wasted cycles.

2.7 Automation today (ADS)

~280 automation-eligible conditions (~25% of claim volume) run through keyword extraction over OCR'd documents plus rules derived from the rating schedule and the M21 manual. It can auto-order exams, ingest VA medical-center records directly, and generate an ARSD (pre-brief for the rater). Notable failure: "Smart Claim Check," an attempt to predict claim success at intake, didn't work — confirming what evidence is present is tractable; proving what's missing (the thing that actually prevents deferrals) is the unsolved problem. This is directly relevant to any AI feature we build: don't repeat that failure mode.

2.8 Known systemic failure modes (from VA's own field guide + internal HCD research)

1. PDF round-trip — structured data serialized to an image and OCR'd back into data. Lossy, slow, acknowledged internally as absurd.
2. Late validation — the deferral loop described above.

3. Siloed intelligence — a front-end ML contention classifier (deployed late 2025) closed the free-text classification gap, but that signal isn't shared with internal automation, which re-derives the same mapping independently.
4. No callback channel to representatives — VSOs and VA duplicate evidence-gathering work with no interface between them.
5. Paper-majority ingress — the worst path is still the most common one.
6. Delivery pressure / technical debt — rushed timelines ship features before requirements settle; combined with 2–3 VSR handoffs per claim, this compounds context-switching cost internally.

2.9 What VA's own veteran research found (HCD study, 60 veteran interviews, Jan 2023)

- Transparency/expectations gap — veterans feel VA's opacity is intentional; decisions reference evidence/exams they don't recognize.
- Fellow veterans are the trusted information source — word of mouth (good or bad) outweighs official channels.
- "Gambling for help" — VSO support quality is inconsistent and veterans have no way to evaluate a VSO in advance.
- Apprehension vs. motivation — filing is emotionally loaded, not just administrative.
- "Needle in a haystack" — veterans don't know what evidence is actually needed, so they over-submit and still get told it's insufficient.
- Stressful exam scheduling — rigid, non-negotiated appointment windows.
- Accessible but incomplete status info — the tracker shows a stage label ("preparation of notification") with no meaningful estimate or explanation.
- Doubtful decision letters — math doesn't visibly reconcile (e.g., combined ratings), and letters don't explain how a rating could change.
- Trust built on personal connection, not scripted call-center interactions.
- Additional key findings: veterans can't tell if their wait time is normal; multiple submission channels create channel-format confusion; unexplained direct deposits arrive before/instead of a decision letter and cause panic; there's no clear channel to report file errors; non-traditional service members (Guard/Reserve) are poorly served by a one-size-fits-all process.

2.10 What VA's own employee research found

- Employees are effective and want to help, but structural issues fight them: inconsistent interpretation of rules across regional offices (NWQ hands a claim to whoever has capacity, with no consistent "order of operations" across stations); production quotas that reward volume over the harder/better-for-the-veteran work; disconnected systems (VBMS doesn't talk to JLV/CPRS, contractor exam data doesn't flow back into VHA records); and training/mentorship that's generic rather than scenario-based.
- Remote work was reported to have measurably increased VSR productivity and retention.

2.11 The system, in current numbers (Aug 2026)

A more recent briefing than the original field guide gives sharper, more current figures worth carrying forward — noting where they update or add nuance to the July 2024 / Jan 2023 sources above:

- 6.34M veterans currently receive disability pay — 36% of all living veterans — for \$174B/year, tax-free, for life.
- 77 days average time for VA to decide a claim, down from 141 in 2023 — meaningfully faster than the "~80+ days" figure in the original field guide, which reflects real, recent improvement.
- Two distinct rework loops exist, not one: the ~40% mid-pipeline deferral rate (Section 2.5, claims bounced back to development before a decision is issued) is a different number from the 1 in 3 decided claims that comes back afterward as a supplemental claim, higher-level review, or Board appeal. VA logged 969,644 such post-decision reviews in FY25 alone. Together, these mean a large share of the system's total effort is rework, at two different points in the pipeline.
- Two-thirds of VA's entire current queue is not first-time filings — it's veterans coming back for increases and secondary conditions once something is already rated. This has a direct product implication: an MVP that only handles original/first-time claims well is optimizing for a third of the actual volume.

2.12 The real bottleneck: VSO capacity, quantified

The free, accredited VSO path is, by every account, the best path for a veteran — and it structurally cannot scale:

- ~8,700 accredited VSO reps for 17.6M veterans.
- 2.5–3.5 hours of manual work to prepare one well-built claim ("the dig": walking the veteran's full history, reading service medical records page by page, checking presumptive eligibility, drafting statements for undocumented conditions). This is why an experienced officer can typically only take on two of these a week — county offices reportedly book four months out, and states average roughly 1.5 claims per officer per day.
- The VSO's accreditation (a public credential) plus the veteran's signed Form 21-22 (POA) is described as "the human API into VA" — it's the only outside key that can submit claims and read a veteran's file inside VBMS. But that human API has no client library: the tooling underneath it is largely early-2010s case-management software, fax machines, and spreadsheets.
- This reframes where the product should focus. The bottleneck isn't VA's internal pipeline (Section 2.4–2.5) so much as the layer before a claim ever reaches VA — intake, the multi-hour "dig," records collection, statement drafting, and packet assembly — which today is almost entirely unowned by any tool. That's consistent with, and sharpens, the MVP direction in Section 4.

2.13 The "dark period" — what happens after submission

Once a claim is submitted, it becomes largely invisible to everyone outside VA: a status bar stuck on a vague label, letters that go unparsed, and no proactive alerts for either the veteran or the VSO. Two data points worth carrying into product decisions: VA reportedly reverses itself on the identical case record in roughly 24% of higher-level reviews (i.e., the same evidence, reviewed twice, produces a different answer a quarter of the time), and one practitioner interview cited a \$70K back-payment error caught only by manual, after-the-fact review. Nobody is systematically watching a claim once it's submitted — for errors,

for missed exams, for silent deadlines, or for an incorrect decision. This is a distinct, well-evidenced product opportunity beyond intake: a claim that is actively monitored through the "dark period," not just filed well at the start.

2.14 Presumptive eligibility is a computable rules engine — not a judgment call

One specific, high-value finding: whether a condition qualifies as a legal presumption (no medical nexus opinion required — see Section 2.6) is, for a wide swath of cases, a deterministic function of facts already printed on a veteran's DD-214:

- Service dates → certain chronic diseases occurring within 1 year of separation are presumed; Gulf-era presumptions require service after Aug 1990.
- Campaign medals (a specific DD-214 field) → e.g., a Vietnam Service Medal or an Iraq/Afghanistan Campaign Medal is, by itself, proof of qualifying theater service.
- Deployment entries → place a veteran in a covered exposure location (burn pits, Agent Orange zones, etc.).
- Job code (MOS/rate) → VA publishes a lookup table of military job codes against likely noise exposure; a match alone concedes acoustic trauma for tinnitus/hearing-loss claims, the single most commonly claimed condition.

In other words, presumptive eligibility for a large share of claims — including PACT Act (burn-pit exposure, 20+ conditions), Agent Orange, and Gulf War illness cohorts — reduces to parsing a DD-214 and joining it against public tables. This is squarely a deterministic, rules-based computation, not a subjective evidentiary judgment — meaningfully different from the general "don't assess evidence sufficiency" guardrail in Section 4.4, and directly relevant to what the orchestrator can safely do in its "coordinator" role (Section 4.3/4.4 have been updated accordingly).

2.15 Illustrative walkthrough: one claim, start to finish

The Aug 2026 briefing includes a detailed, composite (not a real individual) walkthrough of a best-case claim — a service member using the pre-discharge filing path with a free VSO — that's useful for team empathy and worth summarizing rather than reproducing in full:

A shoulder injury goes unreported for years because the service member doesn't want to look weak or risk deployability — years later, this becomes the one condition that gets denied, purely because no record exists ("document everything before you leave" is universally cited as the #1 piece of VSO advice, and almost nobody follows it). A transition class gives a VSO about 15 minutes to make the pitch to file early — most separating service members never take it. The VSO intake session — "the dig" described in Section 2.12 — takes 2.5–3.5 hours and is where most of the actual claim gets built. Submission is described as "a one-way door": once filed, outside parties can only watch and respond, never directly act. Weeks of silent "development" follow, an exam gets scheduled via a phone call from an unknown number (missing it typically means a denial), and a decision eventually arrives as a paper envelope — sometimes preceded by an unexplained direct deposit that the veteran has no context for. The undocumented shoulder is denied; everything else is granted. The 1-year post-decision window opens up a further round of supplemental filing.

The counterfactual — a veteran who instead waits years to file, after separation — loses more than a document trail: back pay only runs from the filing date, not the date the injury occurred, so years of an already-earned rating (worth \$20K+/year at a 70% rating) simply never existed. Same injuries, same law — the only variable is when, and how well, the evidence got assembled. That gap is effectively the product's reason to exist.

3. Product Vision

A veteran-first, conversational AI orchestrator that carries a veteran through the disability claims process — connected directly to VSO tools and services — instead of a veteran having to navigate a fragmented, opaque system alone.

The veteran has exactly one relationship in this product: a conversation with the AI orchestrator. Behind that conversation, the orchestrator connects to and works through the infrastructure VSOs already use (open/public VA APIs, and VSO case-management tooling wherever it's accessible), so the veteran never has to understand VBMS, tracked items, or the eFolder to get a coherent experience. This directly answers the two structural failures from Section 1: it gives the veteran and their VSO a shared, synchronized view of the claim (closing the "no shared visibility" gap), and it pushes plain-language, claim-type-aware guidance to the front of the process instead of leaving validation to happen silently at rating, four VSRs later (softening, though not eliminating, the "late validation" problem — see Section 4.3 for what's actually in scope for MVP).

Positioning relative to reference products:

- Like Assured's front end: a guided, conversational claimant experience — but where Assured's AI does deterministic case-processing work for an insurance adjuster, ours deliberately does not do that job. It coordinates; it doesn't adjudicate (see Section 4.4).
- Unlike PageAI: we're veteran-facing, not backend-only, and we're not trying to replace the VSO's expert judgment or case-management software — we sit on top of it and feed it, positioning VSOs as a channel and partner rather than a customer we're disintermediating.
- Unlike VeteranHQ: we're not an education/content funnel that hands off to a VSO and stops — the orchestrator stays with the veteran through the actual claim, in an ongoing working relationship with the VSO the veteran is already routed to.

3.1 Resolved product decisions (as of this session)

Question	Decision
Primary user	Veteran. Veteran-facing, veteran-first. The product connects directly into VSO tools/services rather than being a VSO-facing tool with a veteran front end bolted on.

Data/integration strategy	Build on top of existing VSO tools and public/open VA infrastructure (Lighthouse APIs, D2D, ARP, etc.) wherever possible, rather than building parallel infrastructure or a new system of record.
Role of AI	Coordinator, not classifier or assessor. The AI does not do evidence classification, diagnostic mapping, or rating/outcome prediction — those stay with VA's existing systems and the VSO's professional judgment. Its job is to navigate the veteran through the process: smoother, faster, less one-sided, than doing it alone.
Go-to-market	VSOs. The product is distributed through and works with accredited Veteran Service Organizations, not sold direct-to-veteran as a paid product or direct-to-attorney like PageAI.
Data/privacy posture	Privacy-by-design, minimal data footprint. The product should avoid holding veteran PII beyond what's operationally required — conceptually, an identifier (possibly hashed) that lets the orchestrator route the right information to the right VSO/VA system, rather than our own copy of the veteran's record. See Section 4.5.

3.2 Design principles derived from the research (carried forward, still valid)

- Validate early, not at rating. Any evidence-completeness guidance should operate at intake/development time, not attempt outcome prediction (VA's ADS already tried this and stalled — see Section 2.7). Consistent with "coordinator, not assessor" — the orchestrator can surface what's needed and nudge, without itself judging sufficiency.
 - Route status through the claim-type lens. Don't build one generic "here's your status" screen — the evidence a claimant needs to see/provide depends entirely on which of the six claim types (Section 2.6) they're in.
 - Close the VSO communication gap. VA structurally cannot notify a rep of a development request; sitting on the veteran's side and relaying to the VSO in real time is a core piece of the orchestrator's job, not a nice-to-have.
 - Be honest about wait times. Veteran research (Section 2.9) repeatedly flags that not knowing whether a wait is "normal" is a top source of anxiety — surfacing expected-duration context (by stage, by claim type) is higher leverage than raw stage labels.
 - Plain language over VA/medical jargon, especially at decision-letter and rating-math explanation points.
 - Move with urgency on the intake + evidence-gap-analysis wedge. PageAI's own site marks "Intake" and "Analysis" as "coming soon" — that's adjacent to, though not identical to, our wedge (their analysis is attorney-facing/appeal-opportunity-focused; ours is veteran-facing/coordination-focused). Assume competitive attention on this space is increasing, not static.
-

4. MVP Definition

4.1 One-line description

A conversational AI orchestrator that a veteran talks to instead of navigating VA.gov, VSO paperwork, and status portals separately — it explains what's happening, gathers what's needed, keeps the veteran's chosen VSO in sync, and does all of this while holding as little identifiable veteran data as the product can structurally get away with.

4.2 The core interaction model

Think of it as one conversation replacing what is today a scattered set of disconnected touchpoints: a VA.gov form, a phone call to a VSO, a mailed letter, a status portal that says "preparation of notification" with no context, and a decision letter full of math that doesn't visibly add up. The veteran has a single ongoing conversational relationship with the orchestrator; the orchestrator is the thing that knows how to translate that conversation into the right action on the VSO/VA side, and translate what comes back from VSO/VA into something the veteran can actually understand and act on.

This is deliberately not a dashboard-first or form-first product. The chat mental model set by the team is that this replaces "a single push of a button" — the veteran shouldn't have to know which button to push, or which of six claim-type lanes (Section 2.6) they're in, or what a tracked item is. They just talk, and the orchestrator figures out what needs to happen and either handles it directly or clearly tells the veteran what's needed and why.

The operating shape, stated precisely: the software does the dig, the veteran confirms, the accredited VSO reviews and files. This phrasing (echoed independently in the Aug 2026 field briefing as the pattern that actually works) matters for two reasons at once:

- It's the legally coherent version of "coordinator." Gathering evidence or filling out forms on someone else's behalf is restricted to accredited individuals (Section 4.4). So the orchestrator's conversational work with the veteran has to be framed as helping the veteran do their own dig and confirm their own answers — not as the orchestrator acting as an unaccredited agent. The accredited VSO remains the one who reviews and actually files. This is also what keeps the product squarely inside the free, accredited channel our GTM already targets, rather than drifting toward the "claim shark" category that just collapsed in court (Section 1.1).
- It has to be fast, not exhaustive. Field data on this is unambiguous: a 15–20 minute intake profile in a real deployment saw 85% abandonment. The target is a conversation that respects a veteran's time — closer to a guided, adaptive dig than a long form — with the software doing the heavy lifting (parsing a DD-214, computing presumptive eligibility per Section 2.14, structuring what's already been said) rather than asking the veteran to supply everything from scratch.

4.2.1 What this looks like end to end

Mapped loosely against the walkthrough in Section 2.15: the veteran talks through their situation; the orchestrator does the mechanical parts of "the dig" (parses identity/service documents, computes presumptive eligibility, structures conditions and dates) and asks the veteran to confirm rather than requiring them to already know what's relevant; a decision-ready draft goes to the partner VSO for review and filing, not to VA directly from us (Section 4.4, Section 5); after submission, the orchestrator becomes

the thing watching the "dark period" (Section 2.13) — translating status, relaying documents and questions between veteran and VSO, flagging deadlines — instead of leaving the veteran to a silent status bar; and when a decision arrives, the orchestrator explains it in plain language, including checking the rating math.

4.3 What the orchestrator does (MVP capabilities)

Capability	What it addresses (tie back to research)	Illustrative concrete feature
Conversational intake & routing — figures out what kind of situation the veteran is in (new claim, increase, presumptive, supplemental, appeal, etc.) and connects them to an accredited VSO to work with	"Needle in a haystack" (Section 2.9); evidence-ladder confusion (Section 2.6); the multi-hour manual "dig" that caps VSO capacity (Section 2.12)	Parse a DD-214 and compute presumptive eligibility automatically (Section 2.14) — a deterministic rules lookup, not a judgment call — then walk the veteran through confirming/adding the rest conversationally
Plain-language process explainer, calibrated to the veteran's specific claim type and current stage — not a generic FAQ	"Accessible but incomplete information" (Section 2.9); jargon/complexity flagged repeatedly in veteran interviews	—
Status translation with honest wait-time context — turns a bare stage label into something like "you're in development, this typically takes X, here's what's happening behind the scenes"	Veterans' top-cited anxiety was not knowing whether a wait was normal (Key Finding 1, Section 2.9); the unwatched "dark period" after submission (Section 2.13)	A "dark-period watcher" — poll status, parse letters, track exam scheduling, surface changes proactively instead of waiting for the veteran to check
Two-way relay between veteran and VSO — when the veteran has a document, question, or update, the orchestrator gets it to the VSO; when the VSO/VA needs something, the orchestrator surfaces it to the veteran in plain language and helps them respond	Directly closes the "no callback channel to representatives" failure mode (Section 2.8) — today, development letters go to the veteran only, never the VSO, even with POA on file	—

Deadline & appointment nudges — reminders tied to suspense dates and exam scheduling	"Stressful exam process" and general anxiety about silent deadlines (Section 2.9); missed exam calls are a cited cause of default denials (Section 2.15)	—
Decision/letter translation — when a decision arrives, explain what it actually means (including reconciling rating math) in plain language	"Doubting decision letters" (Section 2.9) — veterans reported real confusion over combined-rating math that technically is documented but isn't legible	A "VA-math calculator" — combined ratings, bilateral factor, effective-date math — as a deterministic correctness primitive; verifying arithmetic is not the same as judging evidence sufficiency (Section 4.4)

4.4 What the orchestrator explicitly does not do (guardrails)

These are as important to the product definition as what's in scope — they're what keeps this a coordination layer instead of an attempt to rebuild VA's adjudication pipeline, and what keeps the product on the right side of the line between "assisting" and "acting on someone's behalf":

- Does not classify contentions or map them to diagnostic codes. VA already runs a front-end ML classifier for this (Section 2.7); duplicating it adds risk without adding value.
- Does not assess evidence sufficiency or predict claim/rating outcomes. This is exactly the failure mode behind VA's own abandoned "Smart Claim Check" (Section 2.7) — confirming what's present is tractable, confirming what's missing enough to guarantee approval is not, and a wrong prediction here actively harms trust. Nuance: this guardrail applies to subjective evidentiary judgment, not to deterministic, rules-based computation — e.g., presumptive-eligibility lookups (Section 2.14) and rating-math verification (Section 4.3) are fixed, publicly documented functions of known inputs, not judgment calls, and are fine for the orchestrator to compute directly.
- Does not gather evidence or fill out forms on the veteran's behalf. This is a hard legal line, not a product preference: doing so for someone else is restricted to accredited individuals under federal law. The orchestrator helps the veteran do their own dig and confirm their own answers (Section 4.2); an accredited VSO remains the one who reviews and files.
- Does not charge veterans for initial-claim help, ever. Federal law bars this outright, and the three largest companies that ignored it were bankrupted/enjoined in 2026 (Section 1.1). Any monetization sits on the VSO/organization side (consistent with the GTM decision in Section 3.1), never as a fee to the veteran for filing an original claim.
- Does not make the legal or clinical judgment calls — evidence strategy, appeal strategy, medical-opinion interpretation. Those stay with the accredited VSO. The orchestrator's job is to make sure the VSO has what they need, faster, and that the veteran isn't left in the dark while it happens.
- Is not a system of record. It does not aim to become a second eFolder. It routes and translates; VBMS/eFolder and the VSO's own case files remain the source of truth (consistent with the privacy posture in Section 4.5).

- Does not replace the VSO relationship or the VSO's software. It integrates with and feeds it — the goal is a better-prepared, better-informed case arriving at the VSO, not a disintermediated VSO.

4.5 Privacy posture (product-level, not architecture yet)

The product goal, as a design constraint from day one rather than something bolted on later: the orchestrator should need to know as little about who the veteran is as possible, while still being able to actually get their claim done.

Conceptually, at the product level (technical implementation is a separate, later exercise):

- The orchestrator's own storage should hold as little directly identifying data as it can — ideally something closer to a routing identifier (potentially a hashed ID) than a full veteran profile.
- The substantive claim content — service records, medical evidence, contentions — should flow through the orchestrator to the VSO/VA systems that are the actual, already-authorized holders of that data, rather than being duplicated and retained on our side as a second copy.
- The veteran should be able to trust that a conversation with the orchestrator is not the same thing as handing a new company their full medical and service history — it's closer to a translator/router sitting between them and the systems (VSO, VA) that already legitimately hold that data.
- This is a genuine differentiator against the two closest competitors: PageAI ingests and stores full C-files for the firms it serves; VeteranHQ collects a registration/profile for its benefit-matching. A minimal-data-footprint posture is both a trust story for veterans and a lighter compliance surface for us.

This principle should shape MVP scoping decisions (Section 4.6) even before there's a concrete technical design for it — flagged as the top open question carried into Section 5.

4.6 MVP scope

In scope for MVP:

- Conversational intake that identifies the veteran's situation and routes them into the correct claim-type lane (Section 2.6) and to a partner VSO — target ≤ 20 minutes of veteran effort, adaptive rather than a long static form (Section 4.2)
- Deterministic presumptive-eligibility computation from parsed identity/service documents (Section 2.14) — in scope precisely because it's a rules lookup, not a judgment call
- Plain-language, stage- and claim-type-aware process explanation
- Status retrieval and translation, watching the "dark period" (Section 2.13) proactively rather than waiting for the veteran to check, with realistic wait-time framing
- Two-way relay of documents/questions/updates between veteran and VSO
- Deadline and appointment reminders
- Decision-letter translation once a decision is issued, including deterministic rating-math verification
- A minimal-data-footprint identity/routing model for the veteran

Near-term technical entry points (product-level, not full architecture — see Section 5): VA's Benefits Intake API (PDF-packet upload into an eFolder) is open today with instant sandbox keys and no approval

gate, and is reportedly how most third-party filings physically land regardless of front end — a realistic near-term integration surface. The Benefits Claims API (programmatic 526EZ filing, POA, status) is gated to accredited organizations, and new consumer-app onboarding was reconfirmed paused as of Aug 25, 2026 — meaning MVP should be built to operate through an accredited VSO partner rather than attempt direct, independent VA filing. ARP (Accredited Representative Portal) is VA's stated target rail — reportedly absorbing all rep traffic and phasing out the older vendor hookup (D2D) within about a year — and is worth designing toward even if the nearest-term bridge is still the incumbent VSO case-management tools (VetPro/VetraSpec).

Explicitly out of scope for MVP (candidates for later):

- Any evidence-sufficiency assessment, rating prediction, or diagnostic/contention classification (guardrail, not just a phasing decision — see Section 4.4)
- Gathering evidence or filling forms on the veteran's behalf rather than helping the veteran do and confirm their own (legal guardrail, Section 4.4)
- Appeal strategy generation or legal-advice-shaped output
- Supporting more than one VSO relationship per veteran at a time
- An attorney/claims-agent-facing product surface (PageAI's segment) — possible later expansion, not MVP
- Replacing or competing with VSO case-management software — MVP integrates with/feeds it
- Direct, independent programmatic filing into VA systems — blocked by the current Benefits Claims API pause; MVP routes through an accredited VSO instead

4.7 What "MVP works" looks like

A veteran opens a conversation, is understood well enough — in well under 20 minutes of their own effort — to be routed to the right claim type and a partner VSO, stays informed in plain language throughout development without having to call anyone or decode a status portal, has their document/question relayed to the VSO without a mailed-letter round-trip, and receives a decision they can actually understand — while the orchestrator itself never becomes a place where a veteran's full medical and service history sits at rest.

Candidate MVP metrics, framed in the units the domain itself already measures in (per the Aug 2026 briefing's suggested scoring approach):

- VSO rep-hours per decision-ready claim — the core bottleneck metric (currently 2.5–3.5 hours manual, Section 2.12); the product's central bet is compressing this.
- Veteran-minutes of effort to reach a decision-ready, VSO-reviewable claim.
- Packets accepted by the VSO without rework — a proxy for whether the "software does the dig, veteran confirms" model (Section 4.2) is actually producing usable output.
- Days to a filed, decision-ready claim from first conversation.

5. Reference Materials Log

Source	What it covers	Status
--------	----------------	--------

VA Claims Field Guide for Engineers	System architecture, trust zones, pipeline, glossary	Synthesized into Section 2
VBA Disability Compensation HCD Final Presentation (Jan 2023)	Veteran + employee insights, journey map, service blueprint, opportunities	Synthesized into Section 2.9–2.10
Claim Submission & Establishment Pathway Map (pptx, v1.8)	Detailed intake pathway diagram + dictionary/acronym guide	Synthesized into Section 2.3
veteranhq.app/guides/va-disability-explained	Veteran-perspective explainer — and revealed to be a live competing product (education, benefit-matching, calculators, paid tier), not just a guide	Reviewed 8/28/26; synthesized into Section 1
Assured (assured.com)	Insurance-claims agentic AI platform — closest structural analog, different vertical	Reviewed 8/28/26; synthesized into Section 1
PageAI (pageai.co)	VA-law-firm backend claims tooling, no front end today — closest direct comparable; roadmap includes intake + analysis stages	Reviewed 8/28/26; synthesized into Section 1
"The Claims Machine" hackathon briefing (compiled 28 Aug 2026)	Current end-to-end claim narrative, hard bottleneck/timing numbers, infrastructure layer-by-layer breakdown, and concrete "what to build" product ideas, drawn from VA/VBA public data plus ~200 practitioner interviews	Reviewed 8/28/26; synthesized into Sections 1.1, 2.11–2.15, 4.2–4.4, 4.6–4.7, 5

Claude artifact (claude.ai/code/artifact/91486d93...)	Team overview artifact referenced in chat	Not fetchable — requires an authenticated/JS-rendered session; static fetch returned only a placeholder page. Recommend the person who created it export or paste its content directly.
VA internal audit of the process (dated)	Mentioned in chat, not attached — recommend adding	Not yet reviewed
"Page 22" process overview doc	Mentioned in chat, not attached — recommend adding	Not yet reviewed

6. Next Steps

1. Team review of Section 4 (MVP Definition) to sanity-check scope and guardrails.
2. Move into technical scoping (data model, integration architecture) as a separate follow-on doc.

Appendix: Acronym & Term Glossary

Compiled from the VA Claims Field Guide, the pathway-map dictionary, the VBA HCD presentation, and terms introduced in this doc.

Forms & document types

Term	Meaning
526	Disability compensation claim form — the flagship request type
527	Pension claim form
530	Burial benefit claim form
686 / 674	Dependency forms (add a spouse, child, or student dependent); the 686 is VA's pilot for fully digital, PDF-free intake

0966	Intent to File — timestamp that locks the effective date so back pay is computed from the lock, not the eventual submission date
0995 / 0996	Supplemental claim form / higher-level review form (the two decision-review lanes)
21-22 (2122)	Power of Attorney grant to a VSO or attorney
4142	Consent form letting VA retrieve records from private third parties
DBQ	Disability Benefits Questionnaire — structured exam-results form clinicians return

Systems & platforms

Term	Meaning
VBMS	Veterans Benefits Management System — Core, Rating, and Awards modules over corporate data; where a claim is officially "established"
eFolder	Per-veteran document repository inside VBMS Core; the official system of record
CorpDB / BEP	Corporate Database and Benefits Enterprise Platform services — the authoritative relational store behind VBMS
NWQ	National Work Queue — nightly, capacity-modeled router that distributes claims across the 57 regional offices
Lighthouse	VA's API gateway/developer platform; hosts the Benefits Intake API, Benefits Claims API, and D2D
D2D	Digits to Digits — integration path VSO/law-office software uses to establish claims automatically
QuickSubmit	Leidos-hosted web upload portal feeding the mail-handling pipeline
ARP	Accredited Representative Portal — successor to the decommissioned SEP (Stakeholder Enterprise Portal)
Central Mail Portal / DMHS	Work-queue interface over everything mailed, faxed, or uploaded (Direct Mail Handling Service)

ADS	Automated Decision Support — the umbrella automation program (IBM-run) covering ~280 automation-eligible conditions
ARSD	Automated Review Summary Document — machine-generated evidence pre-brief for the rater
JLV / CPRS	VA health-record systems (Joint Legacy Viewer / Computerized Patient Record System) — cited in employee research as poorly integrated with VBMS
BDN	Retired mainframe-era benefits system; its jargon still permeates VBA vocabulary
VDC	VONAPP Direct Connect — legacy back-end service for automatic claim establishment
Exam contractors (QTC, VES, Optum, MSLA)	Private vendors VA contracts with to conduct compensation exams; their structured DBQ output is what raters actually rate from

Claim process & evidence terms

Term	Meaning
AutoCEST	Automatic Claim ESTablishment — a machine, rather than a human, establishes the claim in VBMS/eFolder
ACE	Acceptable Clinical Evidence — rate from existing records, skip the exam
FDC	Fully Developed Claim — claimant certifies the package is complete; development is skipped
Duty to assist	VA's statutory obligation to obtain federal records and identified private records on the claimant's behalf
Tracked item	One outstanding evidence dependency on a claim; collectively, the claim's state machine
Suspense date	The timeout on a tracked item, typically 30 days
Deferral	A rater bouncing a claim back to development for more evidence — the dominant rework loop (~40% of claims that reach rating)
POA	Power of Attorney — authorization for a VSO/attorney to act on a veteran's behalf
VASRD	VA Schedule for Rating Disabilities — the rating rulebook

M21	The adjudication procedures manual — the business-rules source of truth
Contention	One claimed condition (e.g., "right knee," "PTSD"); a claim is a set of contentions
C-file	The veteran's full claims file (used in the legal/attorney context, e.g., by PageAI)
DD-214	The root identity/service document issued at separation — dates, discharge type, job code, medals — from which presumptive eligibility can be computed (Section 2.14)
Nexus	The medical link between a current condition and an in-service event/injury — the third of the three things every claim must prove, alongside a current diagnosis and an in-service event
STR	Service Treatment Records — the in-service medical record DoD transfers to VA once a claim opens
BDD	Benefits Delivery at Discharge — filing 180–90 days before separation; exams happen while still in uniform, decision follows ~30 days after the DD-214 is issued, back pay runs from day one
IDES	Integrated Disability Evaluation System — the separate, joint DoD/VA process for service members found medically unfit for duty (medical retirement), related to but distinct from the standard claims pipeline
TAP	Transition Assistance Program — the mandatory pre-separation class where a VSO typically gets a brief window to pitch filing early
MOS	Military Occupational Specialty (a service member's job code) — one of the DD-214 fields VA uses to compute presumptive eligibility (e.g., noise-exposure lookup tables for tinnitus/hearing loss)
Presumptive condition	A condition where the medical nexus is waived by statute once qualifying service is shown — a deterministic eligibility computation, not a judgment call (Section 2.14)

Roles & organizations

Term	Meaning
VSO	Veteran Service Organization — accredited, free representation (external)
VSR	Veteran Service Representative — internal development-stage claims processor

RVSR	Rating Veteran Service Representative — sets service connection and disability percentage
SVSR	Senior Veteran Service Representative
CA	Claims Assistant
LAS	Legal Administrative Specialist
MSC	Military Service Coordinator
PCR	Public Contact Representative
VSCM / AVSCM	Veterans Service Center Manager / Assistant VSCM
VBA	Veterans Benefits Administration — the benefits arm of the VA
VHA	Veterans Health Administration — the VA's health system
OCTO	Office of the Chief Technology Officer — owns VA.gov and the digital-intake experience
PA&I	Office of Performance Analysis and Integrity — owns claims-processing metrics, reports to Congress
VCIP	Veterans Claims Intake Program — runs the Vendor Claims Intake function
GCIO	GovernmentCIO — prime contractor over Mail Management Services
IBM	Runs the Mail Automation / RPA program under contract
RO	Regional Office (57 nationwide)

Statute, policy & compensation terms

Term	Meaning
PACT Act	2022 statute creating presumptive conditions for toxic exposure — a major automation driver
MST	Military Sexual Trauma — claims route to a dedicated processing center
TDIU	Total Disability based on Individual Unemployability — pays at the 100% level when service-connected conditions prevent substantially gainful work, distinct from a schedular 100% rating

SMC	Special Monthly Compensation — higher tiers above the standard 100% rate for the most serious situations
ADC	Average Days to Complete — VA's headline claims-processing latency metric

Technical / delivery terms used in this doc

Term	Meaning
HCD	Human-Centered Design — the research methodology behind the VBA veteran/employee insights study
RPA	Robotic Process Automation — bots that operate systems the way a human would
OCR	Optical Character Recognition — converts scanned/PDF images back into machine-readable data
FNOL	First Notice of Loss — insurance-claims term (Assured); the intake-equivalent moment in that vertical
ABA Formal Opinion 512	American Bar Association guidance requiring lawyers to verify AI outputs and maintain professional judgment — cited by PageAI as a compliance anchor
SOC 2 Type II / HIPAA	Security/compliance certifications cited by PageAI; relevant benchmarks if we ever sell to attorneys or handle protected health information

Frontend / UX HLD

Veteran App — Frontend High-Level Design

1. Starting assumptions

Stated explicitly rather than left as open questions, consistent with how we closed out the requirements doc:

- **Platform: mobile-first responsive web app (PWA-capable), not a native app store install.** A veteran should be able to tap a link from a VSO, a TAP class QR code, or a text message and be in the conversation within seconds — no app-store gate, no account wall before value is delivered. Camera access (document capture), push notifications (reminders/updates), and offline resume are all achievable in this form factor without the distribution friction of an app store.
 - **Primary device: phone.** Design for a one-handed, thumb-reachable phone screen first; the same layout scales up to tablet/desktop but nothing is designed desktop-first.
 - **Identity is deferred, not front-loaded.** In line with the privacy posture in the requirements (Section 4.5), the veteran can start the conversation before any formal identity/account step — identity is collected only when it's actually needed (routing to a specific VSO), not as a signup wall before the product proves its value.
 - **One orchestrator, one voice, one persistent access point.** There's a single conversational entry point available from anywhere in the app — the product should never make a veteran feel "stuck" in a flow with no way to just ask a question, which is a direct response to the "trust built on personal connection, not scripted interactions" research finding.
-

2. Design principles

Principle	What it means on screen	Where it comes from
Conversation-first, not conversation-only	Chat is the primary engine, but structured, tappable cards (document upload, confirmation summaries, status timelines) live inline in the conversation and as a small set of persistent "home" surfaces for return visits. Never a long static form.	MVP Section 4.2: "not a dashboard-first or form-first product"

Respect the effort ceiling	Every flow is chunked, skippable, and resumable. The app does the parsing (documents, dates, codes); the veteran mostly confirms rather than types from scratch.	MVP Section 4.2.1: a 15–20 minute static intake form saw 85% abandonment in the field data behind this doc
Confirm, don't dictate	Anything the AI infers or drafts is shown back to the veteran as an editable, confirmable card — never silently submitted on their behalf.	Legal guardrail, Section 4.4: gathering evidence/filling forms <i>on someone's behalf</i> is accreditation-gated; the veteran must do and confirm their own
Zero unexplained jargon	Every VA term, acronym, or status label gets a plain-language translation inline, on first use, every time — not just once in a glossary.	Veteran research, Section 2.9: "accessible but incomplete information," decision-letter confusion
Radical honesty about time and about data	Wait times shown as ranges with context ("most claims like yours take about X weeks"), never a bare stage label. A visible, persistent answer to "what do you know about me" available anytime, not just at onboarding.	Veteran research Key Finding 1 (Section 2.9); privacy posture (Section 4.5)
Never block the path to a human	The partner VSO's name, credentials, and contact path are visible and reachable from the claim home screen at all times — the app augments that relationship, it never hides it behind itself.	Veteran research Insight 9 (Section 2.9): trust is built on personal connection, not scripted interaction
Calm, not urgent	No countdown timers, no artificial scarcity, no pressure copy. Deadlines are shown factually with enough lead time to act.	Direct contrast to the predatory "claim shark" pattern the market just legally rejected (Section 1.1)

Accessible by default	Large-text and high-contrast modes, voice input, full screen-reader support, and tolerance for low-bandwidth/older devices are baseline requirements, not stretch goals — this population is disabled by definition and skews older than a typical consumer app audience.	Veteran demographics (requirements Appendix / Section 2.9 research sample: ages 26–90+)
Never promise an outcome	The app never states or implies odds of approval, a predicted rating, or "you'll qualify for X" beyond what's a deterministic, legally defined fact (e.g., presumptive eligibility).	Direct guardrail from VA's own failed "Smart Claim Check" (Section 2.7, Section 4.4)

3. Information architecture

Three persistent surfaces after first run, reachable from a simple bottom navigation (mobile) — deliberately minimal, not a sprawling dashboard:

1. **Talk** — the orchestrator conversation. Default landing screen. Where intake happens and where a veteran can always ask a question, at any point in the claim's life.
2. **My claim** — the ongoing status hub: timeline, what's needed from the veteran, updates/messages with the VSO, upcoming deadlines, and the decision when it arrives.
3. **You** — privacy/data controls, VSO contact info, notification preferences, accessibility settings, help.

First-run flow (not a persistent tab — a one-time linear sequence that deposits the veteran into "My claim" once complete):

Welcome & trust → The dig (conversation) → Review & confirm → Connect to VSO → (*lands in*) My claim

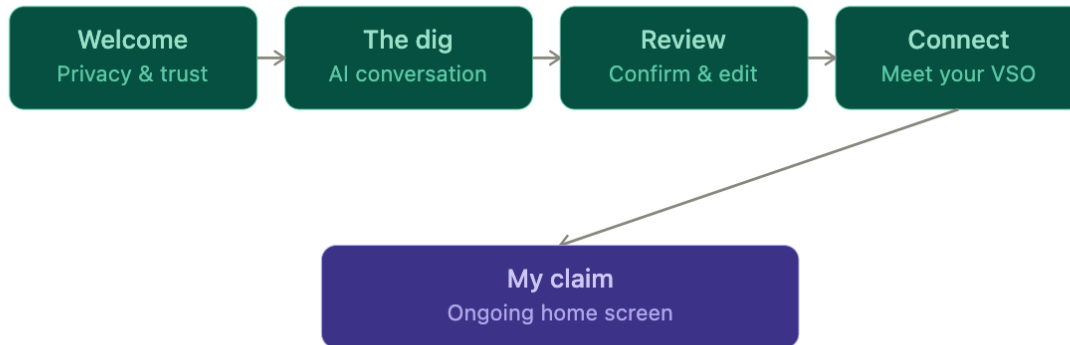


Figure 1. First-run flow

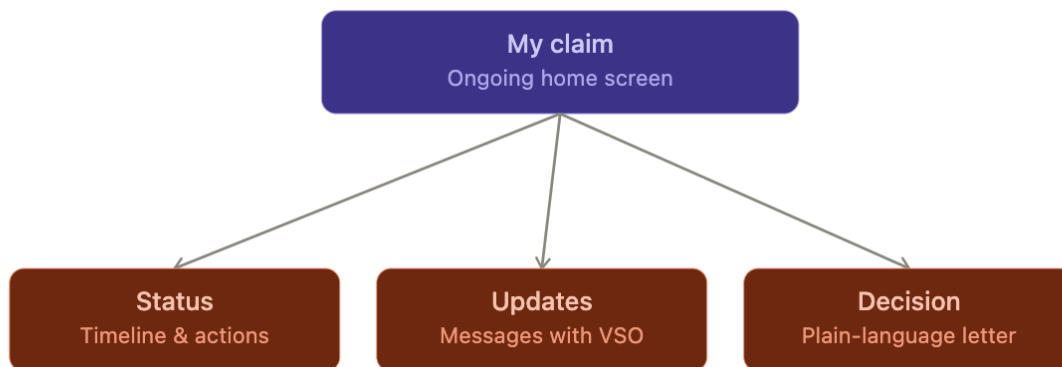


Figure 2. Breakdown of the Home Screen

This is exactly what the two diagrams above show: a short, linear onboarding sequence funneling into a single ongoing hub, which itself branches into three functional areas (status/actions, updates/relay, decision).

4. Screen-by-screen design

4.1 Welcome & trust (*first run only*)

Purpose: Immediately counter the two biggest trust barriers the research surfaced — fear of being scammed (claim sharks, Section 1.1) and fear of losing control of medical/service data (Section 4.5) — before asking for anything.

Content:

- One sentence, plain language: *"A free guide to help you file your VA claim, working with a real accredited Veteran Service Officer. We never charge you, and we're not a law firm."*
- One sentence on data: *"We don't keep a copy of your medical records. We help you gather what you need and hand it to your VSO securely."*
- No account creation required to begin. A single "Let's get started" CTA drops straight into conversation.

Explicitly avoided: any urgency language, any "limited time," any request for payment info, any long terms-of-service wall before first value.

4.2 The dig (*conversational intake*)

Purpose: Replace the 2.5–3.5-hour manual VSO "dig" (Section 2.12) with a fast, adaptive conversation that does the mechanical work so the human VSO doesn't have to.

Core pattern: a chat thread where the AI asks open, low-pressure questions ("Tell me a bit about your service and what's been bothering you since") interleaved with **inline structured cards** that appear only when they add value:

- **Document capture card** — "Got your DD-214 handy? Snap a photo and I'll pull the details." Camera or file picker, inline in the thread.
- **Confirmation card** — after parsing, a compact summary (name, branch, service dates, discharge type, medals, job code) with per-field edit affordances. The veteran taps "Looks right" or corrects a field — never re-types what's already legible from the document.
- **Presumptive eligibility card** — "Based on your service dates and deployment, you may automatically qualify for: [tinnitus — noise-exposure job code match] [burn-pit conditions — PACT Act deployment match]. No extra proof needed for these." This is the deterministic rules-engine output (Section 2.14) surfaced as a confirmable fact, not a suggestion — clearly distinguished in copy from anything requiring judgment.
- **Statement builder** — for undocumented conditions (the "shoulder that was never reported" pattern from the walkthrough, Section 2.15), a short guided prompt sequence that assembles a personal statement, with an option to send a link to a witness for a buddy statement.
- **Soft progress indicator** — a collapsible checklist ("Getting to know you → Checking automatic eligibility → Anything else? → Review"), not a percentage bar, so it informs without feeling bureaucratic.
- **Save and resume anywhere** — closing mid-conversation is a normal, supported action; a reminder (push/SMS/email, per notification preference) brings the veteran back to exactly where they left off.

- **Voice input toggle**, available throughout, for accessibility and for anyone who finds typing on a phone slow or difficult.

4.3 Review & confirm

Purpose: The explicit checkpoint where the veteran confirms their own dig before it goes to a human — the product-level embodiment of the "software does the dig, the veteran confirms, the VSO reviews and files" model (Section 4.2).

Content: A clean, grouped summary — service info; conditions grouped into "qualifies automatically" vs. "needs evidence"; statements; uploaded documents — each section with a direct edit-link back into the relevant point in the conversation. Explicit, unambiguous consent copy: *"You're confirming this is accurate. [VSO name] will review it next."* One clear CTA: **Confirm & send to my VSO.**

4.4 Connect to VSO

Purpose: Introduce the specific accredited human now responsible for the claim, and set an honest expectation for what happens next — directly serving the "trust and personal connection" finding (Section 2.9, Insight 9).

Content: VSO/rep name, organization, and accreditation; plain-language timeline ("They'll typically review this within X days and may reach out with questions"); direct contact information; explicit reassurance that a real, credentialed person — not a black box — is now handling the claim.

4.5 My claim — status home (*the main return screen*)

Purpose: Answer "where do things stand" honestly at a glance, and actively watch the "dark period" (Section 2.13) that today leaves both veteran and VSO in the dark after submission.

Content:

- **Current stage, translated** — never VA's raw label. E.g., *"Your claim is being reviewed (this stage is called 'development'). Most claims like yours take 6–10 more weeks. We'll tell you the moment anything changes."*
- **Visual timeline:** Submitted → Under review → Exam (if applicable) → Decision, current position highlighted, each stage tappable for a one-line plain-language explainer.
- **"Needs your attention"** — translated tracked items, each with a single tappable action (upload a document, e-sign a release, message the VSO) rather than a passive notice.
- **Upcoming** — exam-call warnings ("VA may call from an unknown number to schedule your exam — save it as 'possible VA call'"), suspense-date countdowns shown with enough lead time to matter, never as a last-minute alarm.
- **Updates feed** — a single chronological thread combining translated VA correspondence, VSO messages, and the veteran's own submitted items. This is the two-way relay capability (Section 4.3) made visible: closing the "letters go to the veteran only, never the VSO" gap from Section 2.4/2.8 by making both directions equally visible in one place.

- **Persistent "Ask" entry point** back into the Talk conversation, always available from this screen.

4.6 Decision translator (*appears when a decision is issued*)

Purpose: Directly resolve "Doubting Decision Letters" (Section 2.9) — the single most consistently cited veteran frustration in the research.

Content:

- Plain headline: overall combined rating and monthly payment amount.
- **Condition-by-condition breakdown:** each claimed condition with its individual rating, granted/denied, and a plain-language reason.
- **"How we got to X%" math walkthrough** — VA's actual combined-rating math (not simple addition) shown step by step, using the deterministic rating-math calculator concept from the requirements (Section 4.3) to resolve the specific "math doesn't add up" confusion the research documented.
- **"What this unlocks"** — a plain list of what the new rating opens up (healthcare, home-loan fee waiver, property-tax exemption, education benefits for dependents) — directly answering the research finding that decision letters never mention this (Section 2.9's "rating is a key, not just a check" framing).
- **If anything was denied or came in low** — a calm explanation that options exist (supplemental claim, higher-level review, Board appeal), described factually with the 1-year back-pay-protection window, and a clear CTA to **talk to your VSO about your options**. The app explains that a fork exists; it never recommends which path to take (guardrail, Section 4.4).

4.7 You (*profile, privacy, and settings*)

Purpose: Keep the trust story alive as an ongoing, always-accessible fact, not a one-time onboarding screen that's never seen again.

Content:

- Plain, persistent explanation of what is and isn't stored (reinforcing Section 4.5's minimal-footprint posture).
- VSO contact info, with an option to reach out directly at any time.
- Notification preferences (push, SMS, email) for reminders and updates.
- Accessibility settings: text size, high-contrast mode, voice input default.
- A simple, unambiguous "delete my data" action — consistent with a genuinely minimal-footprint posture, this should be an easy, honest control, not a dark-pattern maze.
- A visible path to reach a human for help, independent of the AI conversation.

5. Traceability: MVP capability → screen

MVP capability (Section 4.3 of requirements)	Primary screen(s)
Conversational intake & routing	The dig (4.2), Review & confirm (4.3)
Plain-language process explainer	Woven throughout — most visible in My claim's stage explainers (4.5)
Status translation with honest wait-time context	My claim — status home (4.5)
Two-way relay between veteran and VSO	My claim — updates feed (4.5)
Deadline & appointment nudges	My claim — upcoming section (4.5), push/SMS notifications
Decision/letter translation	Decision translator (4.6)
Minimal-data-footprint identity model	Deferred identity (Section 1 assumption), You — privacy controls (4.7)

Every MVP capability from the requirements doc has a clear home; nothing in this design introduces a screen that isn't traceable back to a specific requirement.

6. Explicitly not in this frontend

Carried directly from the requirements doc's MVP guardrails (Section 4.4, Section 4.6) into design decisions:

- **No VSO-facing interface.** The VSO continues to work in their existing tools (VetPro/VetraSpec, ARP); this frontend is veteran-only. The "Confirm & send to my VSO" action is the handoff point — what happens on the VSO's side is a backend/integration concern, not a screen in this app.
 - **No appeal-strategy or legal-advice UI.** The Decision screen surfaces that options exist; it never recommends one.
 - **No payment or billing UI anywhere.** The app never charges a veteran for initial-claim help — this is a legal guardrail, not a business decision (Section 4.4).
 - **No sprawling dashboard.** Three surfaces, not a portal with a dozen sections — the whole point is that a veteran shouldn't have to learn a new bureaucracy to escape the old one.
 - **No multi-VSO management.** One veteran, one VSO relationship, for MVP.
 - **No outcome predictions.** No "your odds of approval," no predicted rating — ever.
-

7. Content & tone guidelines

- Second person, active voice, plain English. Define any unavoidable VA term inline, every time it appears — never assume a returning veteran remembers a definition from three weeks ago.
- Calm confidence, not hype. State uncertainty honestly ("most claims like yours take about...") rather than false precision.
- Avoid both extremes of military-adjacent tone: no over-corporate insurance-marketing voice, and no forced patriotic kitsch. Respectful, direct, human.
- Never state or imply a claim outcome before VA decides it.
- Every notification (push/SMS/email) is informational, never urgent-feeling — no red badges, no "act now" framing, consistent with the "calm, not urgent" principle.

8. Accessibility baseline

- WCAG 2.1 AA as the minimum bar, not a stretch target.
 - Full screen-reader labeling on every inline card and widget, not just static text.
 - Color is never the only signal for status (pair with icon/text, not just red/green).
 - Text resizing that doesn't break layout; a high-contrast mode.
 - Voice input as a first-class alternative to typing throughout the conversation.
 - Designed to remain usable on older/lower-powered phones and slower connections, given the rural/lower-bandwidth share of the veteran population noted in the research (Section 2.9 demographics).
-

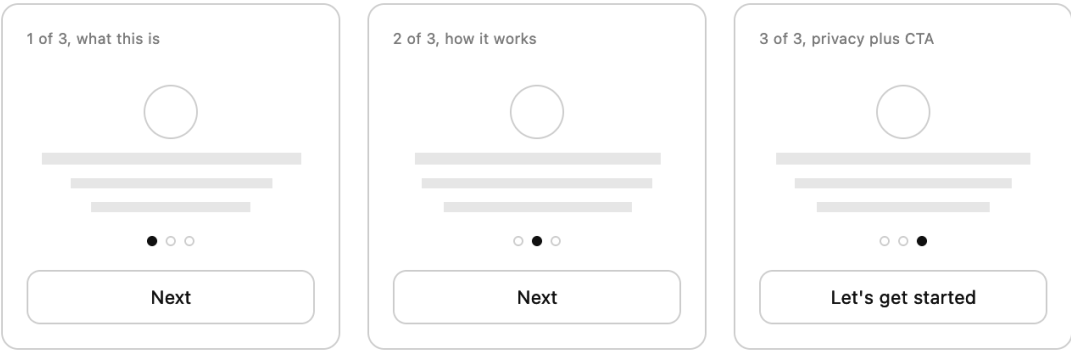
9. Suggested next step

This document defines the shape and content of the frontend; it deliberately stops short of visual design (color system, typography, component styling) and backend/technical architecture, both called out as separate follow-on work in the requirements doc. Natural next steps, in order:

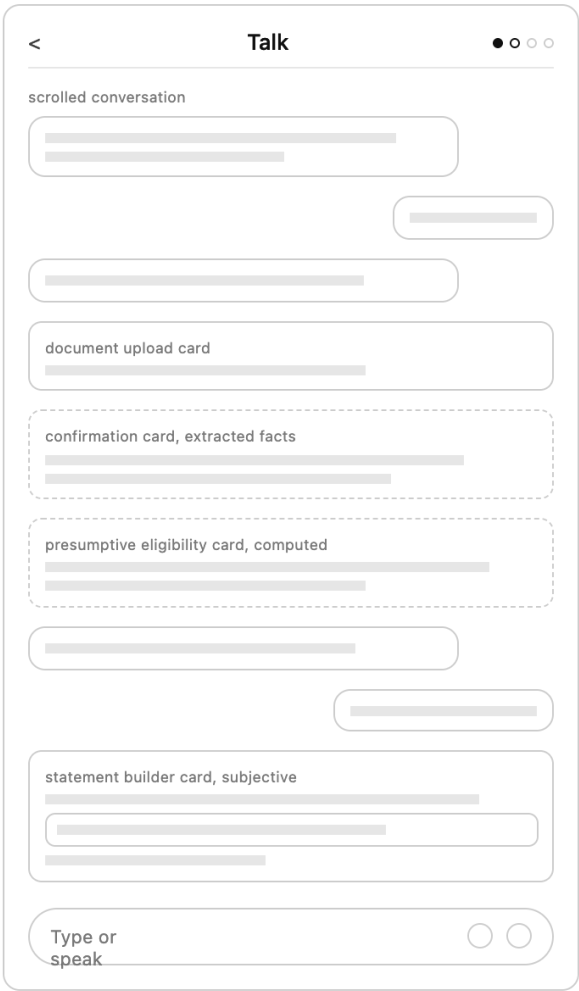
1. Low-fidelity wireframes for the seven screens above, to validate layout and content density before visual design.
2. A visual design system pass (color, type, component library) once the wireframes are validated.
3. Technical/backend architecture (data model, VSO integration, identity/routing) as its own document, per the requirements doc's Next Steps.

Low Fidelity Wireframes

Wireframe 0. First-run flow



Wireframe 1. Talk Page



Wireframe 2. Review, Send, and Connect to my claim

Review

service info

edit

conditions,
automatic

edit

conditions, needs
evidence

edit

documents

edit

Confirm and send to my VSO

Connect

what happens next

Continue to my claim

Wireframe 3. My claim - Submitted + Development Phase

state: just submitted

My claim

nothing needed from you right now

updates

Talk

My claim

You

state: in development

My claim

needs your attention

updates

Talk

My claim

You

Wireframe 4. My claim - Exam + Resolved Phase

state: exam scheduled

My claim

exam scheduled

updates

Talk

My claim

You

state: resolved

My claim

View your decision

updates

Talk

My claim

You

Wireframe 5. My claim - Partial and Full Grant Decision

state: partial grant

Decision

rating math

one condition was denied

Talk to your VSO

Talk

My claim

You

state: fully granted

Decision

rating math

what this unlocks

Talk

My claim

You

Wireframe 6. My claim - Denied Decision

state: fully denied

Decision

your options

Talk to your VSO

Talk

My claim

You

Wireframe 7. You Page (main)

You

your data, at a glance

See full detail

VSO contact

>

Help

>

Notifications

>

Accessibility

>

Delete my data

>

Talk

My claim

You

Wireframe 8. What we store - Full details (from You page)

< What we store

your routing ID

your service and medical records

your conversation with your guide

☐ Read our full privacy approach

Wireframe 9. Data delete page

request screen

< Delete my data

this does not affect

Delete my data

one step confirm, no maze

Cancel

Yes, delete everything

Wireframe 10. Accessibility + Notification page

< Notifications

☒

☐

☒

< Accessibility

A- A A+

☐

☐

☐

Visual Design Plan

Veteran App — Visual Design

1. Color palette

Color	Hex	Use
Background	#FAF8F5	Page background
Surface	#FFFFFF	Cards
Border	#E4E1DA	Dividers, card borders
Text secondary	#6B6862	Supporting text
Text primary	#2B2A28	Body text
Accent tint	#E3EEEC	Accent backgrounds
Accent	#2B6E63	Primary actions, active nav, links
Success	#4B7A4E	Granted conditions, resolved states
Warning	#B8862E	Needs-attention items
Danger	#B4483B	Delete flow, denied conditions
Computed	#6B6FA0	Deterministic/system-calculated content

2. Typography

Attribute	Spec
Typeface	Public Sans, system-font fallback
Base size	17–18px
Line height	1.6–1.7
Weights	Regular, medium only

3. Shape & icons

Attribute	Spec
Card radius	10–12px
Button / row radius	8px
Borders	1px, neutral
Icons	Outline style, consistent stroke weight
Shadows	None

4. Wireframe convention → visual treatment

Wireframe convention	Real visual treatment
Dashed border (computed fact)	Small "computed" tag, solid border
Solid border (action / subjective content)	Plain neutral card, no tag
Timeline dot, done/current	Accent fill
Timeline dot, upcoming	Neutral outline
Danger-red row (Delete, denied)	Danger color, reserved — no other use
Bottom nav active tab	Accent color + medium weight, no pill/underline

Frontend LLD

Veteran App — Frontend Low-Level Design

1. Tech stack decision

Layer	Choice	Version (current stable, Aug 2026)	Why
Framework	Next.js, App Router	16.3.x (Active LTS)	Default choice for React PWAs in 2026; built-in service-worker/PWA tooling, stable App Router, Turbopack as default bundler, React Compiler support (removes most manual memoization)
UI runtime	React	19.2	Ships with Next.js 16; required for App Router
Styling	Tailwind CSS	v4	Utility-first, pairs directly with the design-token tables in the Visual Design Plan (Section 4 below maps every token)
Component layer	shadcn/ui (on Radix primitives)	latest CLI	Not a dependency — components are copied into the repo and owned outright, which matters here because this app's visual language (Section 4) is specific and non-default; shadcn gives accessible, unstyled-enough primitives to restyle directly rather than fighting an opinionated library. Radix chosen over the newer Base UI option for broader current ecosystem maturity.
Global client state	Zustand	latest	Minimal boilerplate for session/claim-cache state; avoids Redux ceremony for an app this size
Server/data state	TanStack Query	latest	Caching, loading/error states, and background refetch for claim status — works identically against the mock layer now and the real API later

Forms/validation	React Hook Form + Zod	latest	Typed schema validation shared between form inputs and the mock/real API contracts (Section 6)
Icons	Tabler Icons (React)	latest	Outline style, matches Visual Design Plan Section 3's icon spec directly
Testing	Vitest + React Testing Library, Playwright	latest	Unit/component tests + real-browser mobile-viewport e2e
Accessibility testing	@axe-core/playwright	latest	Automated WCAG checks wired into e2e runs, not a manual-only process
Deployment	Vercel	—	Same maintainer as Next.js; fastest path to a live, installable PWA tomorrow. Revisit only if a data-residency requirement emerges later (unlikely given the minimal-footprint architecture already decided in the requirements doc).

Known platform constraint to build around from day one

Research confirms: iOS Safari only delivers push notifications to a PWA that has been explicitly added to the Home Screen — a browser tab can never receive push on iOS, permission or not. This directly affects Wireframe 10 (Notifications) and the reminder/deadline features in My claim:

- The app must include an explicit "Add to Home Screen" prompt/flow, not just a notification-permission prompt.
- SMS and email must be treated as equally primary channels, not fallbacks — this was already the design (Wireframe 10 has all three as separate toggles), and this research confirms that decision was correct, not just nice-to-have.
- Push permission should only be requested after install, in response to a direct user tap (Safari requirement), never on page load.

2. Project scaffolding

```
npx create-next-app@latest veteran-app --typescript --tailwind --app --turbo
cd veteran-app
npx shadcn@latest init
npx shadcn@latest add button card badge switch dialog input textarea avatar tabs
```

```
npm install zustand @tanstack/react-query zod react-hook-form @hookform/resolvers
npm install @tabler/icons-react
npm install -D vitest @testing-library/react @testing-library/jest-dom
```

```
npm install -D @playwright/test @axe-core/playwright
npm install next-pwa
```

3. Folder structure

```
veteran-app/
├── app/
│   ├── (onboarding)/          # Wireframe 0 — no bottom nav, linear flow
│   │   ├── welcome/page.tsx
│   │   ├── review/page.tsx    # Wireframe 2
│   │   └── connect/page.tsx    # Wireframe 2
│   ├── (main)/                # Post-onboarding — has bottom nav
│   │   ├── layout.tsx         # renders <BottomNav />
│   │   ├── talk/page.tsx      # Wireframe 1
│   │   ├── claim/
│   │   │   ├── page.tsx       # Wireframe 3/4 — My claim, all states
│   │   │   └── decision/page.tsx # Wireframe 5/6 — Decision translator, all states
│   │   └── you/
│   │       ├── page.tsx       # Wireframe 7
│   │       ├── what-we-store/page.tsx # Wireframe 8
│   │       ├── delete/page.tsx  # Wireframe 9 (request + confirm as local state, not two routes)
│   │       ├── notifications/page.tsx # Wireframe 10
│   │       ├── accessibility/page.tsx # Wireframe 10
│   │       ├── vso-contact/page.tsx
│   │       └── help/page.tsx
│   ├── layout.tsx             # root layout, fonts, providers
│   ├── manifest.ts            # PWA manifest
│   └── globals.css
├── components/
│   ├── ui/                    # shadcn primitives, restyled per design tokens
│   ├── chat/
│   │   ├── ChatThread.tsx
│   │   ├── MessageBubble.tsx
│   │   ├── DocumentUploadCard.tsx
│   │   ├── ConfirmationCard.tsx # "computed" tag styling
│   │   ├── EligibilityCard.tsx  # "computed" tag styling
│   │   ├── StatementBuilderCard.tsx
│   │   ├── ProgressChecklist.tsx
│   │   ├── ResumeBanner.tsx
│   │   └── ChatInputBar.tsx     # text + voice toggle
│   ├── claim/
│   │   └── StageBanner.tsx
```

- StageTimeline.tsx
- NeedsAttentionCard.tsx # warning tint
- UpcomingCard.tsx
- UpdatesFeed.tsx
- ResolvedBanner.tsx
- decision/
 - RatingHeadline.tsx
 - ConditionRow.tsx # success/danger tag variants
 - RatingMathCard.tsx # "computed" tag styling
 - UnlocksCard.tsx
 - OptionsCard.tsx # denied-state primary CTA
- you/
 - DataSummaryCard.tsx
 - SettingsRow.tsx
 - VsoCard.tsx # reused from onboarding Connect screen
 - DeleteConfirmDialog.tsx
- nav/
 - BottomNav.tsx
- shared/
 - AccentButton.tsx
 - ComputedTag.tsx # the one shared component implementing Section 4's dashed→tag

rule

- StatusTag.tsx # success / warning / danger variants
- Toggle.tsx
- lib/
 - api/
 - client.ts # single swap point — see Section 6
 - mock/ # fixtures + mock handlers
 - types.ts # shared domain types
 - store/
 - sessionStore.ts # Zustand — routing ID, not PII (Section 6)
 - accessibilityStore.ts # Zustand — text size, contrast, voice default
 - tokens.ts # design tokens as JS constants, mirrors tailwind.config
- public/
 - icons/ # PWA icon set, all required sizes
 - sw.js # generated by next-pwa
- tailwind.config.ts
- e2e/ # Playwright specs, one per wireframe number
- test/ # Vitest component specs

4. Design tokens → Tailwind config

Directly implements the Visual Design Plan's four tables. This is a literal config, not a translation exercise the coding agent needs to redo.

```
// tailwind.config.ts
export default {
  theme: {
    extend: {
      colors: {
        background: '#FAF8F5',
        surface: '#FFFFFF',
        border: '#E4E1DA',
        'text-secondary': '#6B6862',
        'text-primary': '#2B2A28',
        'accent-tint': '#E3EEEC',
        accent: '#2B6E63',
        success: '#4B7A4E',
        warning: '#B8862E',
        danger: '#B4483B',
        computed: '#6B6FA0',
      },
      borderRadius: {
        card: '12px',
        control: '8px',
      },
      fontFamily: {
        sans: ['Public Sans', 'system-ui', 'sans-serif'],
      },
      fontSize: {
        base: ['17px', { lineHeight: '1.65' }],
      },
      fontWeight: {
        normal: '400',
        medium: '500',
      },
    },
  },
}
```

Enforced rule, not just convention: only AccentButton, ComputedTag, and StatusTag (Section 3) are allowed to reference accent, computed, success, warning, or danger directly. Every other component consumes color through those three shared components. This is what keeps the Visual Design Plan's "reserved meaning" rule (Section 4 of that doc) enforceable in code rather than just documented.

5. Routing map

Route	Wireframe	Layout group	Notes
/welcome	0	onboarding	3-card sequence as local component state, not 3 routes
/review	2	onboarding	
/connect	2	onboarding	includes VSO-matching transitional state
/talk	1	main	default landing route post-onboarding
/claim	3, 4	main	one route, four states driven by claim status from the data layer, not four routes
/claim/decision	5, 6	main	one route, three states (partial/full/denied) driven by decision data
/you	7	main	
/you/what-we-store	8	main	
/you/delete	9	main	request + confirm are local component state (a two-step form), not two routes — matches the wireframe's actual interaction, which never reloaded the page between steps
/you/notifications	10	main	
/you/accessibility	10	main	
/you/vso-contact	—	main	referenced by You page map, no numbered wireframe — build from the VsoCard pattern already established in Wireframe 2
/you/help	—	main	referenced by You page map, no numbered wireframe

Root / redirects to /welcome if no session exists, /talk if a session exists but onboarding is incomplete, /claim if onboarding is complete — this logic lives in app/layout.tsx via the session store, not scattered across pages.

6. Data layer and mock API

Every screen is built against this layer, never against fetch calls scattered in components. This is the single most important section for "pick up and go" — it means frontend work does not block on a backend that doesn't exist yet, and swapping to the real API later is a one-file change.

6.1 Domain types (lib/api/types.ts)

```
type RoutingId = string; // never a name, SSN, or file number — Section 4.5 of requirements
```

```
type ClaimType = 'original' | 'increase' | 'presumptive' | 'supplemental' | 'higher-level-review' | 'fdc';
```

```
type ClaimStage = 'submitted' | 'development' | 'exam-scheduled' | 'resolved';
```

```
type ConditionOutcome = 'granted' | 'denied' | 'pending';
```

```
interface Condition {  
  id: string;  
  name: string;  
  outcome: ConditionOutcome;  
  rating?: number; // 0-100, present once decided  
  computedEligible: boolean; // true = deterministic/presumptive, drives the "computed" tag  
}
```

```
interface Claim {  
  routingId: RoutingId;  
  claimType: ClaimType;  
  stage: ClaimStage;  
  vso: VsoInfo;  
  conditions: Condition[];  
  needsAttention: AttentionItem[];  
  upcoming: UpcomingItem[];  
  updates: UpdateEntry[];  
  decision?: Decision;  
}
```

```
interface Decision {  
  combinedRating: number;  
  monthlyAmount: number;  
  conditions: Condition[];  
  unlocks: string[];  
}
```

```
interface VsoInfo {  
  name: string;
```

```

organization: string;
accreditationId: string;
contactMethods: { type: 'phone' | 'message' | 'email'; value: string }[];
}

// Chat message union — mirrors the assembled Talk wireframe exactly
type ChatMessage =
| { type: 'ai-text' | 'veteran-text'; text: string }
| { type: 'document-upload'; prompt: string }
| { type: 'confirmation-card'; fields: { label: string; value: string }[] }
| { type: 'eligibility-card'; conditions: Condition[] }
| { type: 'statement-builder'; prompt: string };

```

6.2 Mock layer (lib/api/mock/)

Ship four fixture claims covering every state the wireframes require, so no screen is ever built against a guess:

Fixture	Covers
claim.justSubmitted.ts	Wireframe 3 empty state
claim.inDevelopment.ts	Wireframe 3 in-progress state
claim.examScheduled.ts	Wireframe 4 exam state
claim.resolvedPartial.ts	Wireframe 4 resolved → Wireframe 5 partial
claim.resolvedFullGrant.ts	Wireframe 5 full grant
claim.resolvedDenied.ts	Wireframe 6 denied

A `MOCK_MODE` env flag switches `lib/api/client.ts` between these fixtures and real fetch calls — build every screen against `MOCK_MODE=true` starting today.

6.3 Client interface (lib/api/client.ts)

```

interface ApiClient {
  getClaim(routingId: RoutingId): Promise<Claim>;
  sendChatMessage(routingId: RoutingId, text: string): Promise<ChatMessage[]>;
  confirmClaimDraft(routingId: RoutingId): Promise<{ vso: VsoInfo }>;
  deleteMyData(routingId: RoutingId): Promise<void>;
}

```

This is the entire contract the rest of the app depends on. When a real backend exists, only `lib/api/client.ts`'s implementation changes — no component in `components/` should ever need to know whether it's talking to mock fixtures or a live API.

7. State management

State	Tool	Lives in
Routing ID (session identity, never PII)	Zustand, persisted to <code>localStorage</code>	<code>sessionStore.ts</code>
Accessibility preferences (text size, contrast, voice default)	Zustand, persisted	<code>accessibilityStore.ts</code>
Claim data, chat messages	TanStack Query, keyed by routing ID	fetches via <code>client.ts</code>
Onboarding progress (which of the 3 Welcome cards, dig progress checklist)	Local component state / URL state	not global — this data has no reason to outlive the flow

Per the requirements doc's privacy posture (Section 4.5): the session store must never hold a name, SSN, file number, or document content. If a component needs to display "Jamie R." or similar, that string comes from the API response for that render, not from anything cached client-side beyond the current query.

8. Device capabilities

Capability	Wireframe	Implementation
Document capture	1 (document upload card)	<code><input type="file" accept="image/*" capture="environment"></code> — reliable across mobile browsers with zero extra permissions UI. Upgrade path to <code>getUserMedia</code> for an in-app camera view is a v2 enhancement, not MVP.
Voice input	1 (chat input bar)	Web Speech API (<code>SpeechRecognition</code>). Full support on Chrome/Edge Android, partial on Safari — must be built as progressive enhancement: hide the mic toggle entirely on unsupported browsers rather than showing a broken control, and text input must always work identically with or without it.

Push notifications	10	Web Push API, gated behind an explicit "Add to Home Screen" step per the iOS constraint in Section 1. Request notification permission only on direct tap of an in-app "Enable notifications" control, never on load.
SMS / email notifications	10	Backend-dependent (not a frontend capability) — frontend only needs to collect and display the preference toggle state via the API client.

9. Accessibility implementation checklist

Directly from the Frontend Design doc's Section 8, made concrete:

- [] Every interactive element reachable via keyboard tab order; verify with Playwright + manual keyboard pass, not just automated tooling
- [] @axe-core/playwright run as part of the e2e suite (Section 10) — zero violations is a merge gate, not a nice-to-have
- [] All status/outcome information (granted/denied/needs-attention) paired with text or icon, never color alone — matches StatusTag's built-in design (Section 3)
- [] Text resizing via the Accessibility screen (Wireframe 10) actually scales layout, not just font-size in isolation — test at the largest setting against every wireframe, especially the denser ones (Talk, Decision translator)
- [] Voice input is additive only — every flow completable via text/tap alone
- [] Screen-reader labels on all icon-only buttons (bottom nav, chevrons, edit pencils)

10. Testing strategy

Layer	Tool	Scope
Component unit tests	Vitest + RTL	Every component in components/, especially ComputedTag/StatusTag/AccentButton since they enforce the color-reservation rule
End-to-end	Playwright, mobile viewport (iPhone and Android device profiles)	One spec per route in Section 5, run against MOCK_MODE=true fixtures
Accessibility	@axe-core/playwright, folded into the e2e specs above	Every route, not a separate audit pass

Visual regression	Playwright screenshot comparison (optional, add once visual design stabilizes)	Deferred — not needed for first build
-------------------	--	---------------------------------------

11. Day-one build order

Sequenced so each phase produces something to demo, and later phases never block on earlier ones being "perfect":

1. Scaffold + tokens — Section 2 commands, Tailwind config (Section 4), shared components (AccentButton, ComputedTag, StatusTag), mock API layer (Section 6) with all six fixtures.
 2. Navigation shell — BottomNav, the two route groups ((onboarding), (main)), root redirect logic.
 3. You section — smallest, most self-contained wireframe set (7, 8, 9, 10) — good first full vertical slice to validate the whole pattern (routing, tokens, mock data) end to end before tackling Talk.
 4. My claim — Wireframes 3–4, all four states driven by the mock fixtures.
 5. Decision translator — Wireframes 5–6, all three states.
 6. Talk — Wireframe 1, the most complex screen (chat thread, five inline card types, voice input, resume banner) — deliberately last among the core screens since it depends on patterns (cards, tags) proven out in Sections 3–5.
 7. Onboarding — Wireframe 0 and 2 — last because it's the shortest, simplest flow and benefits from every component already existing by this point.
 8. PWA polish — manifest, service worker, install prompt, push permission flow (Section 1's iOS constraint).
 9. Accessibility and e2e pass — Sections 9–10 run against the fully assembled app.
-

12. What this LLD deliberately does not cover

- Real backend/API implementation — Section 6's mock layer stands in until it exists.
- Visual assets beyond icons (illustrations, if any are added to Wireframe 0) — flagged in the Visual Design Plan as a future decision, not blocking.
- Analytics/monitoring — not specified anywhere upstream in the requirements doc; add only if/when a requirement emerges, not preemptively.
- The exact accent hue's final sign-off — Visual Design Plan Section 7 already flagged this as the one open judgment call; the token is wired in Section 4 above and trivially swappable if the team changes it.

Frontend Deep Dives

Veteran App — Deep Dives: AI Conversation and Document Scanning

1. AI-powered conversation (Talk page)

1.1 Frontend responsibility boundary

The frontend owns: rendering streamed AI text, rendering the specific inline cards (document upload, confirmation, eligibility, statement builder) when the backend signals them, loading/typing/error states, and the message protocol contract. It does not own model selection, prompting, conversation orchestration, or the deterministic computations (document parsing, presumptive-eligibility rules) the AI calls as tools — those are backend/AI-orchestration work.

This boundary is also where a requirements-doc guardrail becomes a concrete architecture decision, not just a policy: the AI is a coordinator, not a classifier or assessor (Section 4.4 of the requirements doc). That means presumptive eligibility and document field extraction must be deterministic backend tools the AI calls, with their structured results rendered directly by EligibilityCard / ConfirmationCard — never the LLM's own free-text prose. The model can talk about the result; it must not be the source of the result.

1.2 Tradeoffs considered

Decision	Options weighed	Choice	Why
Model/provider	Claude, GPT, Gemini, or a specific single vendor	Provider-agnostic frontend, model choice deferred to backend/product	Not a frontend decision to make unilaterally, and shouldn't need to be — the frontend should be built so a provider swap or later multi-provider strategy is a backend-only change.
Client/server SDK	Hand-rolled fetch + manual SSE parsing vs. a streaming toolkit	Vercel AI SDK (ai package)	It's the de facto standard for exactly this Next.js pattern in 2026 — provider-agnostic across Claude/GPT/Gemini via a one-line swap, native token-by-token streaming, and native Zod-validated tool calling, which is precisely the mechanism needed to trigger the inline cards reliably. Hand-rolling this is real, unnecessary risk given a mature, well-supported library already fits the stack.

How cards get triggered	AI free-text that the frontend parses for intent vs. structured tool calls	Structured tool calls only	Parsing free text for "should a card appear here" is fragile and exactly the kind of thing that quietly breaks. Tool calls with Zod schemas map directly onto the ChatMessage union type already defined in the LLD — a tool result is a ConfirmationCard payload, not something inferred from prose.
Transport	WebSocket vs. streaming HTTP	Streaming HTTP (AI SDK's built-in data stream protocol)	The conversation is mostly one-directional (AI streaming out, veteran sending discrete messages in) — full-duplex WebSocket complexity isn't earning its keep here.

1.3 Implementation strategy

Client side: the AI SDK's useChat hook backs ChatThread (already scaffolded in the LLD's components/chat/ folder). It exposes both text deltas (for streaming AI prose into MessageBubble) and tool-call/tool-result events (for rendering DocumentUploadCard, ConfirmationCard, EligibilityCard, StatementBuilderCard) from the same stream — no separate polling or parsing layer needed.

Server side (contract, not implementation): a single streaming API route the frontend calls, exposing backend tools with Zod schemas:

```
// Tool contracts the frontend renders against — implementation is backend work
parseDocument(file) → ExtractedField[] // renders as ConfirmationCard
computeEligibility(serviceInfo) → Condition[] // renders as EligibilityCard, deterministic — not LLM prose
saveStatement(text) → { saved: true } // renders as confirmation within StatementBuilderCard
```

Loading states: token-by-token text streaming gives natural "typing" feedback for free. Tool calls that take longer (document parsing in particular) need their own in-card loading state — ConfirmationCard shows a skeleton while its tool call is in flight, matching the awaiting-parse state already defined in Section 2 below.

Error handling: AI or network failures degrade to a calm, specific retry prompt in the thread itself (consistent with the "calm, not urgent" tone principle) — never a raw error dump. Conversation history persists client-side so a retry doesn't lose context, consistent with the resume-banner pattern already built for The dig.

Guardrail enforcement, concretely: EligibilityCard and ConfirmationCard must only ever render from a tool-result payload, never from a chat message's free-text content. This is enforceable in code — those two components simply shouldn't accept a plain-text prop for their primary content, which makes the guardrail a type-level constraint, not just a design intention.

1.4 Forward-looking notes

- Because the frontend speaks to one internal streaming endpoint rather than directly to a specific provider's API, the model can be changed or multi-sourced later without touching frontend code.
 - Voice output (the AI speaking back, not just transcribing input) would extend this same architecture — explicitly out of scope for MVP, not architecturally blocked later.
-

2. Document scanning and capture

2.1 What tool this actually is

The tool is the native browser file input, not a third-party SDK:

```
<input type="file" accept="image/*" capture="environment">
```

No document-scanning library, no camera SDK, no WASM decoder — deliberately. This was a tradeoff in v1.0 (a live in-app camera view was considered and set aside as effort not justified by MVP need); this section makes the choice's real limitations explicit, since "just use the file input" undersells what actually has to be handled around it.

2.2 Concrete limitations of this tool

Limitation	What it actually means	Mitigation
No live capture guidance	No viewfinder overlay, no in-the-moment blur/glare/framing feedback — the veteran points, shoots, and only finds out afterward if it's usable	Fast server-side parse feedback loop (Section 2.4) with a calm, specific retake prompt, rather than trying to replicate capture guidance client-side
HEIC format on iOS, and this is the one that actually breaks something	Many iPhones capture in HEIC/HEIF by default. The Canvas API — the same API this app's compression step relies on — cannot decode HEIC at all; attempting it throws a decode error. Safari's behavior when transcoding HEIC to JPEG is documented as inconsistent depending on the input's accept attribute and whether multiple files are selected — it cannot be relied on	Detect HEIC by file signature before attempting compression. If detected: skip client-side compression entirely and upload the original file with a flag; the backend must handle HEIC decoding, since the frontend structurally cannot

No enforced size/resolution ceiling at capture	A modern phone camera can produce 10MP+ images (10MB+ before compression)	Client-side compression (non-HEIC path) resizes to a 2000px long edge — well beyond OCR's actual needs — before upload
Desktop fallback behavior	The capture attribute is a mobile-only hint; desktop browsers just show a plain file picker with no capture guidance at all	Acceptable for MVP given the mobile-first assumption in the Frontend Design doc, but worth knowing this isn't a mobile-only app in practice — some veterans will use a desktop
No control over source filename	iOS assigns filenames that follow device-specific patterns	Replace with a generated identifier client-side before upload — the original filename is never sent (Section 2.3)

2.3 Exact data sent to the backend

POST /api/documents

Content-Type: multipart/form-data

routingId: string — session identifier only, never a name/SSN/file number

documentType: 'dd214' | 'medical-record' | 'other'

file: Blob

— non-HEIC path: re-encoded JPEG, 2000px max long edge, ~80% quality

— HEIC path: original file, unmodified, flagged via captureMetadata below

captureMetadata: { originalFormat: string, compressedClientSide: boolean }

Explicitly never sent:

- The original device-assigned filename — replaced with a generated identifier before upload.
- EXIF metadata, on the non-HEIC path — this is a real privacy property of the compression step, not just a size optimization. The Canvas API's toBlob()/toDataURL() re-encode from raw pixel data only; EXIF (including GPS coordinates, camera/device identifiers, and capture timestamp) has no mechanism to survive that redraw. Compressing the image and stripping its metadata happen as one unavoidable step, which is a genuine trust point worth stating on the What we store screen (Wireframe 8) once built — not just an engineering footnote.
- On the HEIC path specifically, this metadata stripping does not happen client-side (Canvas can't touch the file at all, per Section 2.2) — the backend must strip it before storage. This is a hard requirement to hand to backend, not optional cleanup, since it's the one path where the frontend's usual privacy guarantee doesn't apply.

One implementation detail worth flagging: EXIF orientation data is also stripped by this process, which means orientation must be read and applied before drawing to the canvas, or the compressed image will display sideways for photos taken in portrait rotation. A real, easy-to-miss bug if the compression step is built naively.

2.4 State machine

idle → capturing → format-check (HEIC? skip to uploading : compress first)
→ uploading → awaiting-parse

awaiting-parse → parsed (→ populates ConfirmationCard as "system-extracted", Section 4)
→ parse-failed (→ calm retake prompt, or manual entry after two failed attempts)

2.5 Forward-looking notes

- If HEIC uploads turn out to be a large share of real traffic (likely, given it's the default capture format on recent iPhones), the next real investment is a client-side HEIC decoder — deliberately not built now, since it's meaningful bundle size and processing cost on exactly the lower-powered devices the accessibility requirements care most about, and there's no usage data yet to justify it.
 - The `documentType` field is already extensible for medical records and buddy statements, which will need this identical pipeline later.
-

3. Backend contracts this depends on

Contract	Purpose	Section
Streaming chat endpoint (AI SDK-compatible)	Powers the Talk conversation, exposes tools below	1.3
<code>parseDocument</code> tool	Deterministic document field extraction, called by the AI, never performed by it	1.1, 1.3
<code>computeEligibility</code> tool	Deterministic presumptive-eligibility rules, same constraint	1.1, 1.3
POST <code>/api/documents</code>	Accepts capture upload; must handle HEIC decoding and EXIF stripping server-side, since the frontend structurally cannot on that path	2.3